

LAPORAN PRATIKUM STRUKTUR DATA

**IMPLEMENTASI ALGORITMA SORTING LANJUTAN (MERGE, QUICK,
SHELL) DALAM JAVA DENGAN VISUALISASI LANGSUNG**

DI SUSUN OLEH :

ABDUL KARIM ALGAZALI

NIM 2511532029

DOSEN PENGAMPU : Dr. WAHYUDI, S.T, M.T

ASISTEN LABORATORIUM : Zahran Azaria Anvaya



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

PADANG

2026

KATA PENGANTAR

Puji syukur kehadiran Tuhan Yang Maha Esa, karena atas rahmat dan izin-Nya laporan praktikum “Implementasi Algoritma Sorting Lanjutan (Merge Sort, Quick Sort, Shell Sort) dalam Java dengan Visualisasi Langsung” ini dapat diselesaikan dengan baik. Saya ucapkan terima kasih sebesar-besarnya kepada Dr. Wahyudi, S.T, M.T selaku dosen pengampu mata kuliah Struktur Data, serta kepada asisten laboratorium yang telah membimbing pelaksanaan praktikum ini.

Laporan ini disusun sebagai bentuk dokumentasi kegiatan praktikum Pekan 8 yang bertujuan untuk memahami dan mengimplementasikan tiga algoritma sorting lanjutan: Merge Sort, Quick Sort, dan Shell Sort. Berbeda dengan praktikum pekan 7 yang fokus pada algoritma sorting dasar (Bubble, Selection, Insertion), praktikum kali ini menekankan pada algoritma dengan kompleksitas yang lebih efisien, khususnya divide and conquer serta optimasi dengan gap (Shell Sort). Pembuatan GUI dengan Swing kembali dikembangkan untuk memvisualisasikan proses Merge Sort dan Bubble Sort secara langkah per langkah, sehingga pemahaman tentang mekanisme pembagian, perbandingan, penggabungan, serta pertukaran elemen menjadi lebih intuitif.

Saya menyadari bahwa penulisan laporan praktikum ini masih jauh dari kata sempurna. Namun, dengan segala kemampuan dan pengetahuan yang dimiliki, saya telah berupaya agar laporan ini dapat selesai dengan baik. Oleh karenanya, saya dengan rendah hati menerima saran, masukan, dan kritikan guna penyempurnaan laporan ini.

Padang, Mei 2026

Penulis

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	3
1.1 Latar Belakang	3
1.2 Tujuan	3
1.3 Manfaat	4
BAB II PEMBAHASAN.....	5
2.1 Konsep Algoritma Sorting	5
2.2 Langkah pengerjaan	6
2.3 Latihan	21
BAB III KESIMPULAN.....	24
3.1 Kesimpulan	24
3.2 Saran	24
DAFTAR PUSTAKA.....	25

BAB I

PENDAHULUAN

1.1 Latar Belakang

Pada praktikum minggu ke-7, kita telah mempelajari tiga algoritma sorting dasar yaitu Bubble Sort, Selection Sort, dan Insertion Sort yang memiliki kompleksitas waktu $O(n^2)$ dalam kasus rata-rata. Meskipun mudah diimplementasikan, algoritma tersebut kurang efisien untuk dataset berukuran besar. Oleh karena itu, pada praktikum minggu ke-8 ini, kita memperkenalkan algoritma sorting lanjutan yang memiliki performa lebih baik, yaitu:

1. Merge Sort – Menggunakan pendekatan divide and conquer: membagi array menjadi dua bagian, mengurutkan masing-masing bagian secara rekursif, lalu menggabungkannya kembali. Kompleksitas waktu $O(n \log n)$ dalam semua kasus.
2. Quick Sort – Juga menggunakan divide and conquer namun dengan memilih elemen pivot dan mempartisi array sehingga elemen lebih kecil di kiri dan lebih besar di kanan. Kompleksitas rata-rata $O(n \log n)$, kasus terburuk $O(n^2)$ jika pivot tidak dipilih dengan baik.
3. Shell Sort – Merupakan generalisasi dari Insertion Sort yang membandingkan elemen dengan jarak tertentu (gap) dan secara bertahap memperkecil gap tersebut. Kompleksitas tergantung urutan gap, dapat mencapai $O(n \log^2 n)$ atau lebih baik.

Selain implementasi command line, praktikum ini juga menuntut pembuatan program visual interaktif untuk Merge Sort dan Bubble Sort (sebagai penguatan dari minggu lalu) menggunakan Java Swing. Sebagai studi kasus akhir, kita akan membuat aplikasi pengurutan playlist lagu berdasarkan judul (ascending) menggunakan Shell Sort, dengan antarmuka berbasis teks (command line) namun menerapkan sorting pada array of objects.

1.2 Tujuan

1. Memahami konsep dan mekanisme tiga algoritma sorting lanjutan: Merge Sort, Quick Sort, dan Shell Sort.
2. Mampu mengimplementasikan ketiga algoritma tersebut dalam bahasa Java untuk tipe data primitif (int).
3. Mampu membuat program visual interaktif dengan Java Swing untuk menampilkan proses Merge Sort langkah demi langkah serta memperbaiki visualisasi Bubble Sort.
4. Memahami perbedaan kompleksitas waktu dan karakteristik setiap algoritma, terutama keunggulan $O(n \log n)$ dibanding $O(n^2)$.
5. Menerapkan sorting pada studi kasus pengurutan data lagu (objek) berdasarkan judul (String) secara ascending menggunakan Shell Sort.
6. Mengelola state algoritma pada Merge Sort yang lebih kompleks (rekursif diubah menjadi antrian langkah).
- 7.

1.3 Manfaat

1. Mahasiswa memahami algoritma sorting yang lebih efisien untuk dataset besar beserta kelebihan dan kekurangannya.
2. Mahasiswa terampil mengimplementasikan algoritma rekursif (Merge Sort, Quick Sort) serta transformasi ke dalam bentuk iteratif untuk visualisasi step-by-step.
3. Mahasiswa mampu membangun antarmuka pengguna untuk visualisasi algoritma lanjutan, termasuk menampilkan proses penggabungan (*merge*) dan penyalinan elemen.
4. Mahasiswa memperoleh pengalaman dalam mengelola state yang lebih kompleks (multi-tahap: merging, copying) di antara event tombol, suatu keterampilan penting dalam pemrograman event-driven.

BAB II

PEMBAHASAN

2.1 Konsep Algoritma Sorting

a. Merge Sort

Merge Sort bekerja dengan strategi *divide and conquer* :

- Divide: Bagi array menjadi dua bagian (kiri dan kanan) hingga tersisa satu elemen.
- Conquer: Urutkan masing-masing bagian secara rekursif.
- Combine: Gabungkan dua bagian yang sudah terurut menjadi satu array terurut menggunakan proses *merge* (membandingkan elemen dari kedua bagian dan menyalin yang lebih kecil ke array sementara).

Kompleksitas waktu: $O(n \log n)$ untuk kasus terbaik, rata-rata, dan terburuk. Stabil dan tidak bergantung pada urutan awal, namun membutuhkan memori tambahan $O(n)$.

b. Quick Sort

Quick Sort juga menggunakan *divide and conquer* namun dengan partisi :

- Pilih elemen *pivot* (misalnya dengan median of three untuk menghindari kasus terburuk).
 - Partisi array sehingga semua elemen $\leq$ pivot berada di kiri pivot, dan yang $\geq$ pivot di kanan.
 - Rekursif terapkan Quick Sort pada subarray kiri dan kanan.
- Kompleksitas rata-rata $O(n \log n)$, kasus terburuk $O(n^2)$ jika pivot selalu elemen terkecil/terbesar. Tidak stabil, tetapi in-place (tidak perlu memori tambahan besar).

c. Shell Sort

Shell Sort adalah peningkatan dari Insertion Sort dengan membandingkan elemen yang berjarak *gap* :

- Mulai dengan $gap = n/2$, kemudian perkecil gap (misal $gap = gap/2$).
 - Lakukan insertion sort untuk setiap sub-array yang berjarak gap .
 - Proses diulang hingga $gap = 1$ (insertion sort biasa).
- Kompleksitas tergantung urutan gap , umumnya $O(n \log^2 n)$ hingga $O(n^{3/2})$. Tidak stabil, tetapi in-place dan lebih cepat dari insertion sort untuk data acak.

Dalam praktikum ini, semua algoritma diimplementasikan untuk mengurutkan integer (command line) dan untuk studi kasus lagu (Shell Sort berdasarkan judul dengan `compareToIgnoreCase` untuk urutan A-Z).

2. QuickSort_2511532029 Program:

```

1 package peham_2511532029;
2
3 public class QuickSort_2511532029 {
4     static void swap_2029(int[] arr, int l_2029, int j_2029) {
5         int temp_2029 = arr[l_2029];
6         arr[l_2029] = arr[j_2029];
7         arr[j_2029] = temp_2029;
8     }
9
10    static void medianOfThree_2029(int[] arr, int low_2029, int high_2029) {
11        int mid_2029 = low_2029 + (high_2029 - low_2029) / 2;
12
13        if (arr[low_2029] > arr[mid_2029]) {
14            swap_2029(arr, low_2029, mid_2029);
15        }
16
17        if (arr[low_2029] > arr[high_2029]) {
18            swap_2029(arr, low_2029, high_2029);
19        }
20
21        if (arr[mid_2029] > arr[high_2029]) {
22            swap_2029(arr, mid_2029, high_2029);
23        }
24        swap_2029(arr, mid_2029, low_2029);
25    }
26
27    static int partition_2029(int[] arr, int low_2029, int high_2029) {
28        medianOfThree_2029(arr, low_2029, high_2029);
29
30        int pivot_2029 = arr[high_2029];
31        int i_2029 = (low_2029 - 1);
32
33        for (int j_2029 = low_2029; j_2029 <= high_2029 - 1; j_2029++) {
34            if (arr[j_2029] < pivot_2029) {
35                i_2029++;
36                swap_2029(arr, i_2029, j_2029);
37            }
38        }
39        swap_2029(arr, i_2029 + 1, high_2029);
40        return (i_2029 + 1);
41    }
42
43    static void quickSort_2029(int[] arr, int low_2029, int high_2029) {
44        if (low_2029 < high_2029) {
45            int pi_2029 = partition_2029(arr, low_2029, high_2029);
46            quickSort_2029(arr, low_2029, pi_2029 - 1);
47            quickSort_2029(arr, pi_2029 + 1, high_2029);
48        }
49    }
50
51    public static void printArr_2029(int[] arr) {
52        for (int i_2029 = 0; i_2029 < arr.length; i_2029++) {
53            System.out.print(arr[i_2029] + " ");
54        }
55        System.out.println();
56    }
57
58    public static void main(String[] args) {
59        int[] arr = {10, 7, 8, 9, 1, 5};
60        int N_2029 = arr.length;
61        System.out.println("Data sebelum diurutkan:");
62        printArr_2029(arr);
63
64        quickSort_2029(arr, low_2029: 0, N_2029 - 1);
65
66        System.out.println("Data terurut quicksort:");
67        printArr_2029(arr);
68    }
69 }

```

Gambar 2.2
Penjelasan program:

Program ini mengimplementasikan Quick Sort dengan optimasi median of three untuk memilih pivot.

- Fungsi medianOfThree_2029 mengambil tiga elemen (low, mid, high), mengurutkannya, dan menempatkan median sebagai pivot di posisi high.
- Fungsi partition_2029 melakukan partisi: semua elemen lebih kecil dari pivot dipindahkan ke kiri, pivot ditempatkan pada posisi yang benar.
- Fungsi quickSort_2029 memanggil dirinya sendiri untuk subarray kiri dan kanan.
- swap_2029 digunakan untuk menukar elemen.

Keunggulan median of three: mengurangi kemungkinan kasus terburuk $O(n^2)$ pada data yang hampir terurut.

Output:

```

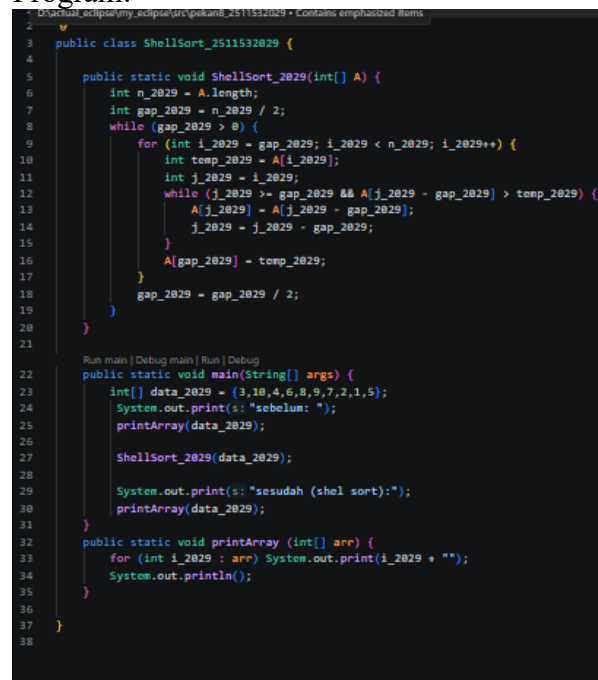
Data sebelum diurutkan:10 7 8 9 1 5
Data terurut quicksort:1 5 7 8 9 10
PS D:\actual_eclipse\my_eclipse\src>

```

Gambar 2.3

3. Program ShellSort_2511532029

Program:



```
1  D:\actual_eclipse\my_eclipse\src\pekan8_2511532029 • Contains emphasized items
2
3  public class ShellSort_2511532029 {
4
5      public static void ShellSort_2029(int[] A) {
6          int n_2029 = A.length;
7          int gap_2029 = n_2029 / 2;
8          while (gap_2029 > 0) {
9              for (int i_2029 = gap_2029; i_2029 < n_2029; i_2029++) {
10                 int temp_2029 = A[i_2029];
11                 int j_2029 = i_2029;
12                 while (j_2029 >= gap_2029 && A[j_2029 - gap_2029] > temp_2029) {
13                     A[j_2029] = A[j_2029 - gap_2029];
14                     j_2029 = j_2029 - gap_2029;
15                 }
16                 A[j_2029] = temp_2029;
17             }
18             gap_2029 = gap_2029 / 2;
19         }
20     }
21
22     Run main | Debug main | Run | Debug
23     public static void main(String[] args) {
24         int[] data_2029 = {3,10,4,6,8,9,7,2,1,5};
25         System.out.print(s: "sebelum: ");
26         printArray(data_2029);
27
28         ShellSort_2029(data_2029);
29
30         System.out.print(s: "sesudah (shell sort):");
31         printArray(data_2029);
32     }
33     public static void printArray (int[] arr) {
34         for (int i_2029 : arr) System.out.print(i_2029 + " ");
35         System.out.println();
36     }
37 }
38
```

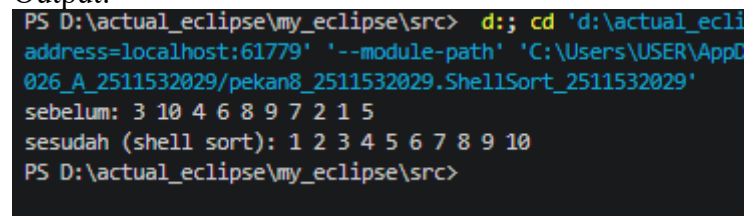
Gambar 2.4

Penjelasan program:

rogram ini mengimplementasikan Shell Sort dengan gap awal $n/2$ dan terus dibagi 2.

- Perulangan while ($gap > 0$) mengontrol pengurangan gap.
- Di dalamnya, for loop dimulai dari $i = gap$ hingga $n-1$, melakukan insertion sort pada elemen-elemen dengan jarak gap.
- Elemen temp disimpan, lalu elemen kiri digeser ke kanan jika lebih besar dari temp.
- Setelah loop selesai, temp ditempatkan pada posisi yang tepat. Perhatikan pada kode Anda terdapat kesalahan: $A[j_2029] = temp_2029$;

Output:



```
PS D:\actual_eclipse\my_eclipse\src> d:; cd 'd:\actual_ecli
address=localhost:61779' '--module-path' 'C:\Users\USER\AppData
026_A_2511532029\pekan8_2511532029.ShellSort_2511532029'
sebelum: 3 10 4 6 8 9 7 2 1 5
sesudah (shell sort): 1 2 3 4 5 6 7 8 9 10
PS D:\actual_eclipse\my_eclipse\src>
```

Gambar 2.5

4. Program BubleSortGUI_2511532029

```
Program / syntax:
package pekan8_2511532029;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.EventQueue;
import java.awt.FlowLayout;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;

public class BubleSortGUI_2511532029 extends JFrame {

    private static final long serialVersionUID = 1L;
    private int[] array_2029;
    private JLabel[] labelArray_2029;
    JButton stepButton_2029;
    private JButton resetButton_2029;
    JButton setButton_2029;
    private JTextField inputField_2029;
    private JPanel panelArray_2029;
    private JTextArea stepArea_2029;

    private int i_2029 = 0, j_2029;
    private boolean sorting_2029 = false;
    private int stepCount_2029 = 1;

    public BubleSortGUI_2511532029() {
        setTitle("Bubble Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        getContentPane().setLayout(new BorderLayout());

        JPanel inputPanel_2029 = new JPanel(new FlowLayout());
        inputField_2029 = new JTextField(30);
        setButton_2029 = new JButton("Set Array");
```

```

        inputPanel_2029.add(new JLabel("Masukkan angka (pisahkan dengan
koma:"));
        inputPanel_2029.add(inputField_2029);
        inputPanel_2029.add(setButton_2029);

        panelArray_2029 = new JPanel();
        panelArray_2029.setLayout(new FlowLayout());

        JPanel controlPanel_2029 = new JPanel();
        stepButton_2029 = new JButton("Langkah Selanjutnya");
        resetButton_2029 = new JButton("Reset");
        stepButton_2029.setEnabled(false);
        controlPanel_2029.add(stepButton_2029);
        controlPanel_2029.add(resetButton_2029);

        stepArea_2029 = new JTextArea(8, 60);
        stepArea_2029.setEditable(false);
        stepArea_2029.setFont(new Font("Monospaced", Font.PLAIN, 14));
        JScrollPane scrollPane = new JScrollPane(stepArea_2029);

        getContentPane().add(inputPanel_2029, BorderLayout.NORTH);
        getContentPane().add(panelArray_2029, BorderLayout.CENTER);
        getContentPane().add(controlPanel_2029, BorderLayout.SOUTH);
        getContentPane().add(scrollPane, BorderLayout.EAST);

        setButton_2029.addActionListener(e -> setArrayFromInput_2029());
        stepButton_2029.addActionListener(e -> performStep_2029());
        resetButton_2029.addActionListener(e -> reset_2029());
    }

    private void setArrayFromInput_2029() {
        String text_2029 = inputField_2029.getText().trim();
        if (text_2029.isEmpty()) return;
        String[] parts_2029 = text_2029.split(",");
        array_2029 = new int[parts_2029.length];
        try {
            for (int k_2029 = 0; k_2029 < parts_2029.length; k_2029++) {
                array_2029[k_2029] = Integer.parseInt(parts_2029[k_2029].trim());
            }
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(this, "Masukan hanya angka
dipisahkan dengan koma!", "Error",
                JOptionPane.ERROR_MESSAGE);
            return;
        }
        i_2029 = 0;
        j_2029 = 0;
        stepCount_2029 = 1;
        sorting_2029 = true;
    }

```

```

stepButton_2029.setEnabled(true);
stepArea_2029.setText("");
panelArray_2029.removeAll();
labelArray_2029 = new JLabel[array_2029.length];
for (int k_2029 = 0; k_2029 < array_2029.length; k_2029++) {
    labelArray_2029[k_2029] = new
JLabel(String.valueOf(array_2029[k_2029]));
    labelArray_2029[k_2029].setFont(new Font("Arial", Font.BOLD, 24));
    labelArray_2029[k_2029].setOpaque(true);
    labelArray_2029[k_2029].setBackground(Color.WHITE);
    labelArray_2029[k_2029].setBorder(BorderFactory.createLineBorder(C
olor.BLACK));
    labelArray_2029[k_2029].setPreferredSize(new Dimension(50, 50));
    labelArray_2029[k_2029].setHorizontalAlignment(SwingConstants.CE
NTER);
    panelArray_2029.add(labelArray_2029[k_2029]);
}
panelArray_2029.revalidate();
panelArray_2029.repaint();
}

```

```

private void performStep_2029() {
    if (!sorting_2029 || i_2029 >= array_2029.length - 1) {
        sorting_2029 = false;
        stepButton_2029.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
        return;
    }
}

```

```

resetHighlights_2029();
StringBuilder stepLog_2029 = new StringBuilder();

```

```

labelArray_2029[j_2029].setBackground(Color.CYAN);
labelArray_2029[j_2029 + 1].setBackground(Color.CYAN);

```

```

if (array_2029[j_2029] > array_2029[j_2029 + 1]) {
    // swap
    int temp_2029 = array_2029[j_2029];
    array_2029[j_2029] = array_2029[j_2029 + 1];
    array_2029[j_2029 + 1] = temp_2029;
}

```

```

labelArray_2029[j_2029].setBackground(Color.RED);
labelArray_2029[j_2029 + 1].setBackground(Color.RED);

```

```

stepLog_2029.append("Langkah ")
    .append(stepCount_2029)
    .append(": Menukar elemen ke-")
    .append(j_2029)

```

```

        .append(" ")
        .append(array_2029[j_2029 + 1])
        .append(" dengan ke-")
        .append(j_2029 + 1)
        .append(" ")
        .append(array_2029[j_2029])
        .append("\n");
    } else {
        stepLog_2029.append("Langkah ")
            .append(stepCount_2029)
            .append(": Tidak ada pertukaran elemen ke-")
            .append(j_2029)
            .append(" dan ke-")
            .append(j_2029 + 1)
            .append("\n");
    }

    stepLog_2029.append("Hasil : ")
        .append(arrayToString_2029(array_2029))
        .append("\n\n");

    stepArea_2029.append(stepLog_2029.toString());

    updateLabels_2029();

    j_2029++;

    if (j_2029 >= array_2029.length - i_2029 - 1) {
        j_2029 = 0;
        i_2029++;
    }

    stepCount_2029++;

    if (i_2029 >= array_2029.length - 1) {
        sorting_2029 = false;
        stepButton_2029.setEnabled(false);
        JOptionPane.showMessageDialog(this, "Sorting selesai!");
    }
}

private void updateLabels_2029() {
    for (int k_2029 = 0; k_2029 < array_2029.length; k_2029++) {
        labelArray_2029[k_2029].setText(String.valueOf(array_2029[k_2029]))
    }
}

private void resetHighlights_2029() {

```

```

        for (JLabel label_2029 : labelArray_2029) {
            label_2029.setBackground(Color.WHITE);
        }
    }

    private void reset_2029() {
        inputField_2029.setText("");
        panelArray_2029.removeAll();
        panelArray_2029.revalidate();
        panelArray_2029.repaint();
        stepArea_2029.setText("");
        stepButton_2029.setEnabled(false);
        sorting_2029 = false;
        i_2029 = 0;
        j_2029 = 0;
        stepCount_2029 = 1;
    }

    private String arrayToString_2029(int[] arr_2029) {
        StringBuilder sb_2029 = new StringBuilder();

        for (int k_2029 = 0; k_2029 < arr_2029.length; k_2029++) {
            sb_2029.append(arr_2029[k_2029]);

            if (k_2029 < arr_2029.length - 1)
                sb_2029.append(", ");
        }

        return sb_2029.toString();
    }

    public static void main(String[] args) {
        EventQueue.invokeLater(new Runnable() {
            public void run() {
                try {
                    BubleSortGUI_2511532029 frame = new
                    BubleSortGUI_2511532029();
                    frame.setVisible(true);
                } catch (Exception e) {
                    e.printStackTrace();
                }
            }
        });
    }
}

```

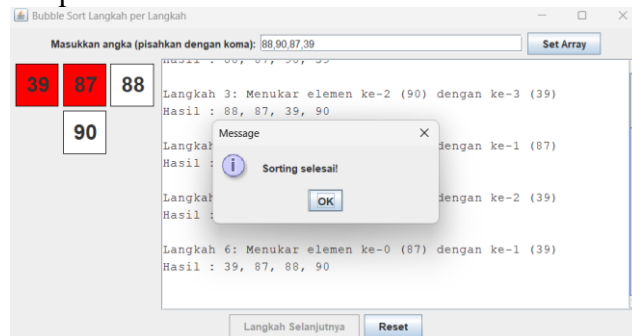
Penjelasan program:

Program ini adalah visualisasi interaktif Bubble Sort menggunakan Java Swing, melengkapi visualisasi Insertion Sort dari minggu lalu. Pengguna dapat memasukkan array angka (dipisah koma), lalu menekan tombol "Langkah Selanjutnya" untuk menjalankan satu demi satu langkah pertukaran.

Komponen utama:

- JTextField untuk input array.
- JButton "Set Array" untuk membaca input dan mereset sorting.
- JPanel berisi JLabel yang merepresentasikan setiap elemen array dengan border dan ukuran tetap.
- JTextArea untuk mencatat log setiap langkah.
- Tombol "Langkah Selanjutnya" memanggil performStep_2029() yang menjalankan satu iterasi dari perulangan dalam Bubble Sort (membandingkan dan menukar jika perlu).
- State disimpan dalam variabel i_2029 (jumlah pass yang selesai) dan j_2029 (posisi perbandingan saat ini).

Output:



Gambar 2.7

5. Program MergeSortGUI_2511532029

Program / syntax:

```
package pekan8_2511532029;
```

```
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.Dimension;
import java.awt.FlowLayout;
import java.awt.Font;
import java.util.LinkedList;
import java.util.Queue;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
```

```

import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import javax.swing.SwingUtilities;

public class MergeSortGUI_2511532029 extends JFrame {

    private static final long serialVersionUID_2029 = 1L;

    private int[] array_2029;
    private JLabel[] labelArray_2029;
    private JButton stepButton_2029, resetButton_2029, setButton_2029;
    private JTextField inputField_2029;
    private JPanel panelArray_2029;
    private JTextArea stepArea_2029;

    private int i_2029, j_2029, k_2029;
    private int left_2029, mid_2029, right_2029;
    private int[] temp_2029;

    private boolean isMerging_2029 = false;
    private boolean copying_2029 = false;
    private int stepCount_2029 = 1;

    private Queue<int[]> mergeQueue_2029 = new LinkedList<>();

    public MergeSortGUI_2511532029() {
        setTitle("Merge Sort Langkah per Langkah");
        setSize(750, 400);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        // Panel input
        JPanel inputPanel_2029 = new JPanel(new FlowLayout());
        inputField_2029 = new JTextField(30);
        setButton_2029 = new JButton("Set Array");
        inputPanel_2029.add(new JLabel("Masukkan angka (pisahkan dengan koma):"));
        inputPanel_2029.add(inputField_2029);
        inputPanel_2029.add(setButton_2029);

        // Panel array visual
        panelArray_2029 = new JPanel();
        panelArray_2029.setLayout(new FlowLayout());

        // Panel kontrol
        JPanel controlPanel_2029 = new JPanel();

```



```

stepButton_2029 = new JButton("Langkah Selanjutnya");
resetButton_2029 = new JButton("Reset");
stepButton_2029.setEnabled(false);
controlPanel_2029.add(stepButton_2029);
controlPanel_2029.add(resetButton_2029);

// Area teks untuk log langkah-langkah
stepArea_2029 = new JTextArea(8, 60);
stepArea_2029.setEditable(false);
stepArea_2029.setFont(new Font("Monospaced", Font.PLAIN, 14));
JScrollPane scrollPane_2029 = new JScrollPane(stepArea_2029);

// Tambahkan panel ke frame
add(inputPanel_2029, BorderLayout.NORTH);
add(panelArray_2029, BorderLayout.CENTER);
add(controlPanel_2029, BorderLayout.SOUTH);
add(scrollPane_2029, BorderLayout.EAST);

// Event Set Array
setButton_2029.addActionListener(e_2029 -> setArrayFromInput_2029());

// Event Langkah Selanjutnya
stepButton_2029.addActionListener(e_2029 -> performStep_2029());

// Event Reset
resetButton_2029.addActionListener(e_2029 -> reset_2029());
}

private void setArrayFromInput_2029() {
    String text_2029 = inputField_2029.getText().trim();
    if (text_2029.isEmpty()) return;

    String[] parts_2029 = text_2029.split(",");
    array_2029 = new int[parts_2029.length];

    try {
        for (int i_2029 = 0; i_2029 < parts_2029.length; i_2029++) {
            array_2029[i_2029] = Integer.parseInt(parts_2029[i_2029].trim());
        }
    } catch (NumberFormatException e_2029) {
        JOptionPane.showMessageDialog(this, "Masukkan hanya angka!",
            "Error", JOptionPane.ERROR_MESSAGE);
        return;
    }

    labelArray_2029 = new JLabel[array_2029.length];
    panelArray_2029.removeAll();

    for (int i_2029 = 0; i_2029 < array_2029.length; i_2029++) {

```

```

        labelArray_2029[i_2029] = new
JLabel(String.valueOf(array_2029[i_2029]));
        labelArray_2029[i_2029].setFont(new Font("Arial", Font.BOLD, 24));
        labelArray_2029[i_2029].setOpaque(true);
        labelArray_2029[i_2029].setBackground(Color.WHITE);
        labelArray_2029[i_2029].setBorder(BorderFactory.createLineBorder(Co
lor.BLACK));
        labelArray_2029[i_2029].setPreferredSize(new Dimension(50, 50));
        labelArray_2029[i_2029].setHorizontalAlignment(SwingConstants.CEN
TER);
        panelArray_2029.add(labelArray_2029[i_2029]);
    }

    mergeQueue_2029.clear();
    generateMergeSteps_2029(0, array_2029.length - 1);

    stepButton_2029.setEnabled(true);
    stepArea_2029.setText("");
    stepCount_2029 = 1;
    isMerging_2029 = false;
    copying_2029 = false;

    panelArray_2029.revalidate();
    panelArray_2029.repaint();
}

private void generateMergeSteps_2029(int left_2029, int right_2029) {
    if (left_2029 < right_2029) {
        int mid_2029 = left_2029 + (right_2029 - left_2029) / 2;

        generateMergeSteps_2029(left_2029, mid_2029);
        generateMergeSteps_2029(mid_2029 + 1, right_2029);

        mergeQueue_2029.add(new int[] { left_2029, mid_2029, right_2029 });
    }
}

private void performStep_2029() {
    resetHighlights_2029();

    if (!isMerging_2029 && !mergeQueue_2029.isEmpty()) {
        int[] range_2029 = mergeQueue_2029.poll();

        left_2029 = range_2029[0];
        mid_2029 = range_2029[1];
        right_2029 = range_2029[2];

        temp_2029 = new int[right_2029 - left_2029 + 1];
    }
}

```

```

i_2029 = left_2029;
j_2029 = mid_2029 + 1;
k_2029 = 0;

copying_2029 = false;
isMerging_2029 = true;

stepArea_2029.append(
    "Langkah " + stepCount_2029++
    + ": Mulai merge dari "
    + left_2029 + " ke "
    + right_2029 + "\n"
);

return;
}

if (isMerging_2029 && !copying_2029) {
    if (i_2029 <= mid_2029 && j_2029 <= right_2029) {
        labelArray_2029[i_2029].setBackground(Color.CYAN);
        labelArray_2029[j_2029].setBackground(Color.CYAN);

        if (array_2029[i_2029] <= array_2029[j_2029]) {
            temp_2029[k_2029++] = array_2029[i_2029++];
        } else {
            temp_2029[k_2029++] = array_2029[j_2029++];
        }

        stepArea_2029.append(
            "Langkah " + stepCount_2029++
            + ": Bandingkan dan salin elemen\n"
        );

        return;
    } else if (i_2029 <= mid_2029) {
        temp_2029[k_2029++] = array_2029[i_2029++];

        stepArea_2029.append(
            "Langkah " + stepCount_2029++
            + ": Salin sisa kiri\n"
        );

        return;
    } else if (j_2029 <= right_2029) {
        temp_2029[k_2029++] = array_2029[j_2029++];

        stepArea_2029.append(

```

```

        "Langkah " + stepCount_2029++
        + ": Salin sisa kanan\n"
    );

    return;

} else {
    copying_2029 = true;
    k_2029 = 0;
    return;
}
}

if (copying_2029 && k_2029 < temp_2029.length) {
    array_2029[left_2029 + k_2029] = temp_2029[k_2029];
    labelArray_2029[left_2029 +
k_2029].setText(String.valueOf(temp_2029[k_2029]));
    labelArray_2029[left_2029 + k_2029].setBackground(Color.GREEN);
    k_2029++;

    stepArea_2029.append("Langkah " + stepCount_2029++ + ":
Tempelkan ke array utama\n");
    return;
}

if (copying_2029 && k_2029 == temp_2029.length) {
    isMerging_2029 = false;
    copying_2029 = false;
}

if (mergeQueue_2029.isEmpty() && !isMerging_2029) {
    stepArea_2029.append("Selesai.\n");
    stepButton_2029.setEnabled(false);
    JOptionPane.showMessageDialog(this, "Merge Sort selesai!");
}
}

private void resetHighlights_2029() {
    if (labelArray_2029 == null) return;

    for (JLabel label_2029 : labelArray_2029) {
        label_2029.setBackground(Color.WHITE);
    }
}

private void reset_2029() {
    inputField_2029.setText("");
    panelArray_2029.removeAll();
    panelArray_2029.revalidate();
}

```

```

        panelArray_2029.repaint();
        stepArea_2029.setText("");
        stepButton_2029.setEnabled(false);
        mergeQueue_2029.clear();
        isMerging_2029 = false;
        copying_2029 = false;
        stepCount_2029 = 1;
    }

    public static void main(String[] args_2029) {
        SwingUtilities.invokeLater(() -> {
            MergeSortGUI_2511532029 frame_2029 = new
            MergeSortGUI_2511532029();
            frame_2029.setVisible(true);
        });
    }
}

```

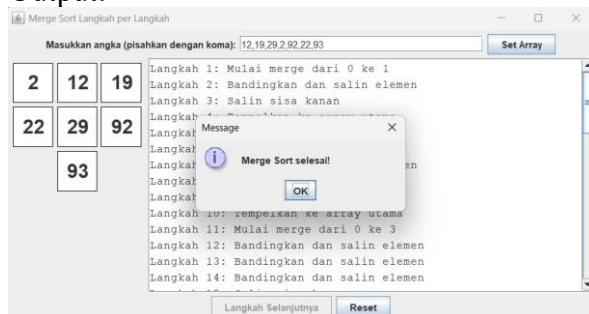
Penjelasan program:

Program ini adalah visualisasi interaktif Merge Sort menggunakan Java Swing. Ini lebih kompleks karena Merge Sort bersifat rekursif. Solusi: semua langkah *merge* (range yang perlu digabung) dihasilkan terlebih dahulu dengan metode `generateMergeSteps_2029` dan disimpan dalam `Queue<int[]>`. Setiap langkah tombol memproses satu langkah kecil dari proses merging.

Komponen utama:

- `Queue<int[]>` `mergeQueue_2029` menyimpan pasangan {left, mid, right} yang harus di-merge.
- State tambahan: `isMerging_2029` (sedang proses perbandingan/salin ke temp), `copying_2029` (sedang menyalin dari temp ke array utama).
- Di `performStep_2029`:
 - Jika tidak merging dan queue tidak kosong, ambil range berikutnya, siapkan temp array, atur indeks `i_2029`, `j_2029`, `k_2029`.
 - Jika merging dan belum copying, lakukan perbandingan dan salin ke temp (dengan sorot warna cyan pada elemen yang dibandingkan).
 - Jika merging dan selesai perbandingan, mulai copying dari temp ke array asli (warna hijau pada elemen yang ditempel).
 - Setelah copying selesai, merge selesai.

Output:



Gambar 2.8

2.3 Latihan

Deskripsi Program:

Program ini adalah aplikasi berbasis teks (command line) untuk mengelola playlist lagu dan mengurutkannya menggunakan Shell Sort berdasarkan judul (ascending). Pengguna dapat menambah lagu (judul, penyanyi, durasi), menampilkan daftar, dan melakukan sorting. Meskipun tidak bergambar, program ini menerapkan sorting pada array of objects dengan comparator berdasarkan atribut judul (String).

1. Program Lagu_2511532029

Program:

```
package pekan8_2511532029;

public class Lagu_2511532029 {
    String judul;
    String penyanyi;
    int durasi; // dalam detik

    public Lagu_2511532029(String judul, String penyanyi, int durasi) {
        this.judul = judul;
        this.penyanyi = penyanyi;
        this.durasi = durasi;
    }
}
```

Gambar 2.8

a) Deskripsi program Lagu_2511532029 :

Kelas Lagu_2511532029 merepresentasikan entitas lagu dengan atribut judul, penyanyi, durasi (enkapsulasi menggunakan default access). Digunakan sebagai objek yang akan diurutkan.

2. Program Sorting_2511532029

Program:

```
package program_2511532029;

import java.util.Scanner;

public class Sorting_2511532029 {
    private Lagu_2511532029[] dataLagu_2029 = new Lagu_2511532029[20];
    private int jumlahLagu_2029 = 0;
    private Scanner scanner_2029 = new Scanner(System.in);

    // Method untuk menambah lagu (input manual)
    public void inputData_2029() {
        if (jumlahLagu_2029 == 20) {
            System.out.println("Playlist sudah penuh (maksimal 20 lagu)!");
            return;
        }

        System.out.print("Masukkan Judul: ");
        String judul = scanner_2029.nextLine();
        System.out.print("Masukkan Penyanyi: ");
        String penyanyi = scanner_2029.nextLine();
        System.out.print("Masukkan Durasi (detik): ");
        int durasi = scanner_2029.nextInt();
        scanner_2029.nextLine(); // consume newline

        dataLagu_2029[jumlahLagu_2029] = new Lagu_2511532029(judul, penyanyi, durasi);
        jumlahLagu_2029++;
        System.out.println("Data Berhasil Ditambah");
    }

    // Method untuk menampilkan semua lagu
    public void tampilData_2029() {
        if (jumlahLagu_2029 == 0) {
            System.out.println("Belum ada data lagu.");
            return;
        }

        for (int i = 0; i < jumlahLagu_2029; i++) {
            System.out.println(i + ". " + dataLagu_2029[i].judul + " - " +
                               dataLagu_2029[i].durasi + " detik");
        }
    }

    // ===== SHELl SORT (urutan berdasarkan DURASI A-Z) =====
    public void shellSort_2029() {
        int n = jumlahLagu_2029;
        // Mulai dengan gap besar, lalu perkecil
        for (int gap = n / 2; gap > 0; gap /= 2) {
            for (int i = gap; i < n; i++) {
                Lagu_2511532029 temp = dataLagu_2029[i];
                int j = i;
                // Bandingkan judul (case insensitive) untuk urutan A-Z
                while (j > 0 && dataLagu_2029[j - gap].judul.compareToIgnoreCase(temp.judul) > 0) {
                    dataLagu_2029[j] = dataLagu_2029[j - gap];
                    j -= gap;
                }
                dataLagu_2029[j] = temp;
            }
        }
        System.out.println("Shell Sort selesai (berdasarkan Judul A-Z)");
    }

    // ===== MENU UTAMA =====
    public void menu_2029() {
        int pilihan;
        do {
            System.out.println("\n===== Sorting Playlist NDM: 2511532029 =====");
            System.out.println("1. Menampilkan Data");
            System.out.println("2. Menu Sorting (Shell Sort - Judul A-Z)");
            System.out.println("3. Exit");
            System.out.print("Pilih menu: ");
            pilihan = scanner_2029.nextInt();
            scanner_2029.nextLine(); // consume newline

            switch (pilihan) {
                case 1:
                    inputData_2029();
                    break;
                case 2:
                    if (jumlahLagu_2029 == 0) {
                        System.out.println("Tidak ada data. Silakan masukkan data terlebih dahulu.");
                        break;
                    }
                    System.out.println("\nData Sebelum Sorting:");
                    tampilData_2029();
                    shellSort_2029();
                    System.out.println("\nData Setelah Sorting (Judul A-Z):");
                    tampilData_2029();
                    break;
                case 3:
                    System.out.println("Terima kasih!");
                    break;
                default:
                    System.out.println("Pilihan tidak valid!");
            }
        } while (pilihan != 3);
    }

    Run event [Main] main [Run] [Main]
    public static void main(String[] args) {
        Sorting_2511532029 program = new Sorting_2511532029();
        program.menu_2029();
    }
}
```

Gambar 2.9

a) Program ini merupakan puncak dari praktikum karena mengintegrasikan:

- Array dataLagu_2029 berukuran maksimal 20.
- Method inputData_2029() untuk menambah lagu dari keyboard.
- Method tampilData_2029() menampilkan daftar lagu (indeks, judul, durasi).
- Method shellSort_2029() mengimplementasikan Shell Sort berdasarkan perbandingan judul.compareToIgnoreCase(temp.judul) > 0 untuk urutan A-Z.
- Method menu_2029() menyediakan antarmuka teks.

Proses Sorting Shell:

- Gap awal = jumlah lagu / 2.
- Untuk setiap gap, lakukan insertion sort dengan jarak gap: bandingkan judul elemen kiri dan kanan, jika kiri lebih besar, tukar (geser).
- Perkecil gap hingga 1.

Output:

```
=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 1
Masukkan Judul: wild
Masukkan Penyanyi: billie
Masukkan Durasi (detik): 209
Data Berhasil Dibuat

=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 1
Masukkan Judul: creep
Masukkan Penyanyi: jabar
Masukkan Durasi (detik): 200
Data Berhasil Dibuat

=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 1
Masukkan Judul: agus
Masukkan Penyanyi: jabur
Masukkan Durasi (detik): 202
Data Berhasil Dibuat

=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 1
Masukkan Judul: hello world
Masukkan Penyanyi: java
Masukkan Durasi (detik): 202
Data Berhasil Dibuat

=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 2

Data Sebelum Sorting:
0. wild - 209 detik
1. creep - 200 detik
2. agus - 202 detik
3. hello world - 202 detik
Shell Sort selesai (berdasarkan Judul A-Z)

Data Setelah Sorting (Judul A-Z):
0. agus - 202 detik
1. creep - 200 detik
2. hello world - 202 detik
3. wild - 209 detik

=== Sorting Playlist NIM: 2511532029 ===
1. Masukkan Data
2. Mulai Sorting (Shell Sort - Judul A-Z)
3. Exit
pilihan: 3
Terima kasih!
PS D:\actual_eclipse\my_eclipse>
```

Gambar 2.10

BAB III

KESIMPULAN

3.1 Kesimpulan

Berdasarkan praktikum implementasi algoritma sorting lanjutan (Merge Sort, Quick Sort, dan Shell Sort) dalam Java, dapat disimpulkan bahwa ketiga algoritma tersebut memiliki kompleksitas waktu $O(n \log n)$ pada kasus rata-rata, sehingga jauh lebih efisien dibandingkan algoritma sorting dasar $O(n^2)$ untuk dataset berukuran besar. Merge Sort stabil dan konsisten namun membutuhkan memori tambahan; Quick Sort dengan optimasi median of three mampu menghindari kasus terburuk dan bekerja in-place; sedangkan Shell Sort memberikan peningkatan signifikan dari Insertion Sort dengan mekanisme gap. Implementasi visual interaktif menggunakan Java Swing berhasil menampilkan proses Merge Sort langkah demi langkah melalui pengelolaan state yang kompleks (antrian merge, proses perbandingan, dan penyalinan), serta melengkapi visualisasi Bubble Sort dari praktikum sebelumnya. Keberhasilan ini membuktikan bahwa algoritma rekursif dapat diubah menjadi eksekusi bertahap dalam pemrograman event-driven dengan perancangan state yang matang.

Selain itu, penerapan Shell Sort pada studi kasus pengurutan playlist lagu berdasarkan judul (ascending) menggunakan array of objects dan comparator `compareToIgnoreCase` menunjukkan bahwa algoritma sorting dapat digeneralisasi ke tipe data kompleks dengan mudah. Program command line yang dibangun memungkinkan pengguna menambah, menampilkan, dan mengurutkan data lagu secara efisien. Dibandingkan dengan algoritma sorting dasar, ketiga algoritma lanjutan ini menjadi fondasi penting dalam pengembangan perangkat lunak yang memerlukan pengurutan data skala besar. Namun demikian, perlu diperhatikan perbaikan bug pada implementasi Shell Sort (kesalahan penempatan nilai temp) serta pengembangan lebih lanjut seperti penyimpanan data ke file dan visualisasi proses divide pada Merge Sort untuk meningkatkan pemahaman mahasiswa.

3.2 Saran

1. Penggunaan visualisasi grafis yang lebih kaya – Untuk Merge Sort, sebaiknya ditambahkan visualisasi proses *divide* (pemecahan array) agar mahasiswa melihat bahwa array dipecah hingga menjadi bagian kecil sebelum digabung. Hal ini dapat dilakukan dengan menampilkan pohon rekursi atau dengan animasi pemisahan blok.
2. Pembagian materi pra-praktikum melalui iLearn tetap perlu dilanjutkan agar mahasiswa memiliki fondasi konseptual yang matang.

DAFTAR PUSTAKA

- [1] Oracle. "Trail: Creating a GUI With Swing". The Java™ Tutorials. 2024. Available:
<https://docs.oracle.com/javase/tutorial/uiswing/>